app.py

```
import os, sys, time
import chromadb
from chromadb.utils import embedding_functions
from fastapi import FastAPI, HTTPException
from openai import OpenAI
from pydantic import BaseModel

app = FastAPI(title="ScalerGPT")

# Fail LOUDLY and clearly if the key is missing - not with a cryptic traceback
API_KEY = os.getenv("OPENAI_API_KEY", "").strip()
if not API_KEY or API_KEY.startswith("sk-paste"):
    sys.exit("[ScalerGPT] OPENAI_API_KEY missing. Put a real key in .env")

llm = OpenAI(api_key=API_KEY)

# The thin client has no built-in embedder - we must supply one explicitly
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key=API_KEY, model_name="text-embedding-3-small")

CHROMA_HOST = os.getenv("CHROMA_HOST", "localhost")
CHROMA_PORT = int(os.getenv("CHROMA_PORT", "8000"))

def connect_to_chroma(retries=30, delay=2):
    # Chroma takes a few seconds to boot. depends_on only waits for its
    # container to START, not to be READY - so we knock politely and retry.
    for attempt in range(1, retries + 1):
        try:
            client = chromadb.HttpClient(host=CHROMA_HOST, port=CHROMA_PORT)
            client.heartbeat()
            print(f"[ScalerGPT] Connected to chroma at {CHROMA_HOST}:{CHROMA_PORT}",
flush=True)
            return client
        except Exception as e:
            print(f"[ScalerGPT] Waiting for chroma ({attempt}/{retries}): {type(e).__name__}",
flush=True)
            time.sleep(delay)
    sys.exit(f"[ScalerGPT] Could not reach chroma at {CHROMA_HOST}:{CHROMA_PORT}")

chroma = connect_to_chroma()
collection = chroma.get_or_create_collection(name="notes", embedding_function=openai_ef)
```

```
class Question(BaseModel):
    query: str

@app.get("/")
def health():
    return {"status": "ScalerGPT is live 📚", "docs_indexed": collection.count(),
            "chroma_host": CHROMA_HOST, "chroma_port": CHROMA_PORT}

@app.post("/ask")
def ask(q: Question):
    if collection.count() == 0:
        raise HTTPException(status_code=400,
            detail="No documents indexed. Run: docker compose exec app python ingest.py")
    # 1. RETRIEVE - find the 3 most relevant chunks
    hits = collection.query(query_texts=[q.query], n_results=3)
    documents = hits.get("documents") or [[]]
    context = "\n\n---\n\n".join(documents[0])
    # 2. AUGMENT - paste those chunks into the prompt
    system_prompt = ("You are ScalerGPT, a helpful teaching assistant. "
        "Answer using ONLY the context below. If it does not contain the answer, "
        f"say you don't know.\n\nCONTEXT:\n{context}")
    # 3. GENERATE - let the LLM write the final answer
    resp = llm.chat.completions.create(model="gpt-4o-mini",
        messages=[{"role": "system", "content": system_prompt},
                  {"role": "user", "content": q.query}])
    return {"question": q.query, "answer": resp.choices[0].message.content,
            "sources_used": len(documents[0])}
```

explain the code one by one

ScalerGPT – Complete Code Explanation (Line by Line)

Your code implements a RAG (Retrieval-Augmented Generation) application using:

- FastAPI – Creates REST APIs.
- ChromaDB – Stores and retrieves document embeddings.
- OpenAI Embeddings – Converts text into numerical vectors.
- OpenAI GPT-4o-mini – Generates answers using retrieved document content.
- Pydantic – Validates incoming API requests.

Let's understand it step by step, from a senior software developer's perspective, using simple English and practical examples.

1. Understand the overall architecture

First, understand what happens when a user asks a question.

ScalerGPT – RAG Architecture

User asks a question

Example: What is Docker?

FastAPI – POST /ask

Receives and validates the question

OpenAI Embedding Model

Converts the question into a vector

ChromaDB – Vector Search

Finds the 3 most relevant document chunks

GPT-4o-mini

Generates an answer using retrieved context

JSON Response to User

This is the RAG workflow:

1. Retrieve
2. Augment
3. Generate

2. Import required libraries

Your code:

```
None
import os, sys, timeimport chromadbfrom chromadb.utils import
embedding_functionsfrom fastapi import FastAPI, HTTPExceptionfrom
openai import OpenAIfrom pydantic import BaseModel
```

Let's understand each import.

Library	Purpose
`os`	Reads environment variables such as API keys
`sys`	Terminates the application when configuration is invalid
`time`	Adds delays between retry attempts
`chromadb`	Connects to the ChromaDB vector database
`embedding_functions`	Provides OpenAI embedding integration
`FastAPI`	Creates REST endpoints
`HTTPException`	Returns HTTP errors
`OpenAI`	Communicates with OpenAI APIs

`BaseModel`	Validates incoming JSON requests

Real-world example:

Suppose a user sends:

```
{
  "query": "What is Docker?"
}
```

FastAPI receives the request, Pydantic validates it, ChromaDB retrieves relevant content, and OpenAI generates the response.

3. Create the FastAPI application

```
app = FastAPI(title="ScalerGPT")
```

Here:

- `FastAPI()` creates an application instance.
- `app` is the application object.
- `title` defines the API documentation title.

When you run:

```
uvicorn main:app --reload
```

FastAPI starts the application.

You can access:

- API documentation: `http://localhost:8000/docs`
- Health endpoint: `http://localhost:8000/`

Interview question: Why do we use FastAPI?

Answer:

FastAPI is a modern Python framework for building REST APIs. It supports asynchronous programming, automatic request validation using Pydantic, and automatically generates Swagger documentation.

4. Read and validate the OpenAI API key

Your code:

```
None
API_KEY = os.getenv("OPENAI_API_KEY", "").strip()
```

Let's break it down.

`os.getenv()`

Reads an environment variable.

For example, your `.env` file might contain:

```
None
OPENAI_API_KEY=sk-proj-xxxxxxxx
```

Important: `os.getenv()` does not automatically load `.env` files. You need to load them using `python-dotenv` or configure the environment variable through Docker.

Why use `.strip()`?

Removes unwanted spaces from the beginning and end.

Example:

```
None
key = "  abc123  "print(key.strip())
```

Output:

```
None
abc123
```

Validate the API key

```
None
if not API_KEY or API_KEY.startswith("sk-paste"):
sys.exit("[ScalerGPT] OPENAI_API_KEY missing. Put a real key in
.env")
```

Two conditions:

1. API key is empty.
2. API key starts with `sk-paste`, which is treated as a placeholder.

If either condition is true, the application terminates.

Why is this useful?

Instead of allowing the application to start and later fail with an authentication error, we validate configuration immediately.

This is called Fail Fast.

5. Initialize OpenAI client

```
None
llm = OpenAI(api_key=API_KEY)
```

This creates an OpenAI client.

You will use this object to communicate with OpenAI models.

For example:

```
None
resp = llm.chat.completions.create(...)
```

Here, `llm` is responsible for sending prompts and receiving generated answers.

Think of it like a Java service:

```
None
OpenAIClient client = new OpenAIClient(apiKey);
```

The Python SDK provides the actual client implementation.

6. Configure the embedding function

```
None
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
api_key=API_KEY,    model_name="text-embedding-3-small")
```

Important RAG concept

This is one of the most important parts of your application.

What is an embedding?

An embedding converts text into a list of numerical values called a vector.

For example:

```
None
Text: Docker is a containerization platform.
```

The embedding model converts it into something conceptually like:

```
None
[0.12, 0.45, -0.32, 0.78, ...]
```

The actual vector has 1,536 dimensions for `text-embedding-3-small` by default.

Why do we need this?

Because computers can compare numerical vectors to identify semantically similar content.

For example:

Question	Document
What is Docker?	Docker is a containerization platform.
Explain containers	Containers package applications and dependencies.

Even though the wording differs, the embedding vectors can be close in semantic space.

Important distinction:

- Embedding model: Converts text into vectors.
- GPT model: Generates human-readable answers.

These are two different AI operations.

7. Configure ChromaDB connection

```
None
CHROMA_HOST = os.getenv("CHROMA_HOST", "localhost")CHROMA_PORT =
int(os.getenv("CHROMA_PORT", "8000"))
```

These variables define where ChromaDB is running.

For local execution:

```
None
CHROMA_HOST=localhost
CHROMA_PORT=8000
```

For Docker Compose, you might have:

```
None
CHROMA_HOST=chroma
CHROMA_PORT=8000
```

Why `chroma`?

Because Docker Compose provides service-name-based communication between containers.

Example:

```
None
services:
  app:
    ...
  chroma:
    image: chromadb/chroma
```

The application container can communicate with the ChromaDB container using its service name.

Important: `localhost` inside a Docker container refers to that same container, not another container.

8. Connect to ChromaDB with retry logic

Your code:

```
None
def connect_to_chroma(retries=30, delay=2):
```

This function establishes a connection with ChromaDB.

Parameters:

- `retries=30`: Try connecting a maximum of 30 times.
- `delay=2`: Wait 2 seconds between attempts.

Why do we need retry logic?

Imagine Docker Compose starts two containers:

1. FastAPI application.
2. ChromaDB database.

Docker may start the FastAPI container before ChromaDB is ready.

Therefore, if FastAPI immediately tries to connect, the connection may fail.

This is why retry logic is required.

Inside the loop

```
None
for attempt in range(1, retries + 1):
```

Executes 30 attempts.

The values are:

```
None
1, 2, 3, 4, ... 30
```

Create the ChromaDB HTTP client

```
None
client = chromadb.HttpClient(    host=CHROMA_HOST,
port=CHROMA_PORT)
```

This creates an HTTP connection client for ChromaDB.

Check whether ChromaDB is responding

```
None
client.heartbeat()
```

This sends a heartbeat request to verify that ChromaDB is reachable.

If successful:

```
None
print(    f"[ScalerGPT] Connected to chroma at
{CHROMA_HOST}:{CHROMA_PORT}",    flush=True)return client
```

`flush=True` ensures the log is written immediately instead of waiting in the output buffer.

Handle connection failure

```
None
except Exception as e:    print(        f"[ScalerGPT] Waiting for chroma ({attempt}/{retries}): {type(e).__name__}", flush=True    )    time.sleep(delay)
```

If connection fails:

- Print the attempt number.
- Print the exception type.
- Wait two seconds.
- Try again.

What if all attempts fail?

```
None
sys.exit(    f"[ScalerGPT] Could not reach chroma at {CHROMA_HOST}:{CHROMA_PORT}")
```

The application terminates.

Retry execution example

1. Attempt 1 – ChromaDB not ready.
2. Wait 2 seconds.
3. Attempt 2 – ChromaDB not ready.
4. Wait 2 seconds.
5. Attempt 3 – ChromaDB responds successfully.
6. Connected
   Return the client.

Maximum theoretical wait: 60 seconds, excluding connection-attempt overhead.

9. Initialize ChromaDB and create a collection

```
None
chroma = connect_to_chroma()
```

Calls the connection function.

Now you have a ChromaDB client.

Next:

```
None
collection = chroma.get_or_create_collection(    name="notes",
embedding_function=openai_ef)
```

Let's understand this.

What is a collection?

A collection in ChromaDB is similar to a table in a relational database.

For example:

MySQL	ChromaDB
Database	Database
Table	Collection
Row	Document record
Columns	Metadata/document fields
Numeric vector	Embedding

Your collection name is:

```
None
notes
```

It stores document chunks and their embeddings.

Why `get_or_create_collection()`?

It performs two operations:

- If the collection already exists, retrieve it.
- Otherwise, create a new collection.

Why pass `embedding_function`?

```
None
embedding_function=openai_ef
```

When you insert text documents, ChromaDB can automatically generate embeddings using OpenAI.

Likewise, when you search using `query_texts`, it generates an embedding for the question.

Important: Your `ingest.py` must use a compatible embedding configuration when inserting documents into this collection.

10. Define the request model using Pydantic

```
None
class Question(BaseModel):    query: str
```

This defines the expected JSON request structure.

For example, valid request:

```
None
{
```

```
  "query": "What is Docker?"
}
```

Invalid request:

```
None
{
  "question": "What is Docker?"
}
```

Why?

Because your model expects a field called `query`, not `question`.

Also:

```
None
{
  "query": 123
}
```

Pydantic will validate the value against the declared string type.

Java comparison:

This is similar to creating a DTO.

```
None
public class Question {
    private String query;
}
```

In your FastAPI application, Pydantic performs request validation automatically.

11. Create the health-check API

Your code:

```
None
@app.get("/")def health():    return {        "status": "ScalerGPT is live 📚",        "docs_indexed": collection.count(),        "chroma_host": CHROMA_HOST,        "chroma_port": CHROMA_PORT    }
```

Let's break it down.

`@app.get("/")`

This is a FastAPI route decorator.

It means:

- HTTP method: GET
- Endpoint: `/`

When someone visits:

```
None
http://localhost:8000/
```

FastAPI executes the `health()` function.

`collection.count()`

Returns the total number of records stored in the collection.

Suppose you indexed 50 document chunks.

Then:

```
None
{
  "status": "ScalerGPT is live 📚",
  "docs_indexed": 50,
  "chroma_host": "chroma",
  "chroma_port": 8000
}
```

This is useful for monitoring the application.

Note: `collection.count()` counts stored records/chunks, not necessarily the number of original PDF files.

12. Create the question-answering API

Core RAG implementation

```
None
@app.post("/ask")def ask(q: Question):
```

This defines a POST endpoint.

When the user sends:

```
None
POST http://localhost:8000/ask
```

With JSON:

```
None
{
  "query": "What is Docker?"
}
```

FastAPI converts the JSON request into a `Question` object.

You access the question using:

```
None
q.query
```

Step 12.1 – Check whether documents exist

```
None
if collection.count() == 0:    raise HTTPException(
status_code=400,        detail="No documents indexed. Run: docker
compose exec app python ingest.py"    )
```

Before searching, check whether the collection contains documents.

If there are no documents:

- Return HTTP status 400.
- Return an understandable error message.

Example:

```
None
{
  "detail": "No documents indexed. Run: docker compose exec app
python ingest.py"
}
```

Why?

There is no point searching an empty vector database.

Step 12.2 – Retrieve relevant document chunks

```
None
hits = collection.query(    query_texts=[q.query],    n_results=3)
```

This is the Retrieval stage of RAG.

Suppose the user asks:

```
None
What is Docker?
```

The application:

1. Converts the question into an embedding.

2. Searches the ChromaDB collection.
3. Finds the three most relevant document chunks.
4. Returns the matching documents and metadata.

Example stored documents:

```
None
Document 1: Docker is an open-source containerization platform.

Document 2: Docker containers package applications and
dependencies.

Document 3: Docker images are used to create containers.

Document 4: Kubernetes manages containerized applications.
```

ChromaDB searches for the three most relevant chunks.

Note that this is semantic similarity search, not simply SQL `LIKE` searching.

Step 12.3 – Extract retrieved documents

```
None
documents = hits.get("documents") or [[]]
```

ChromaDB returns a dictionary containing results.

A simplified example:

```
None
hits = {    "documents": [        [            "Docker is a
containerization platform.",            "Docker containers package
applications.",            "Docker images create containers."
]    ]}
```

Why do we have nested lists?

Because ChromaDB supports multiple queries in one request.

The outer list represents queries.

The inner list represents matching documents for each query.

Therefore:

```
None
documents[0]
```

Gets the matching documents for the first query.

The expression:

```
None
hits.get("documents") or [[]]
```

Provides an empty nested list if documents are missing or empty.

Step 12.4 – Combine the retrieved content

```
None
context = "\n\n---\n\n".join(documents[0])
```

This combines the retrieved chunks into one string.

Example:

```
None
Docker is a containerization platform.

---

Docker containers package applications and dependencies.

---

Docker images are used to create containers.
```

Why use `---`?

It separates individual document chunks and makes the context easier to read.

This combined text is passed to GPT.

13. Augment – Build the system prompt

```
None
system_prompt = (    "You are ScalerGPT, a helpful teaching
assistant. "    "Answer using ONLY the context below. If it does
not contain the answer, "    f"say you don't
know.\n\nCONTEXT:\n{context}")
```

This is the Augmentation stage of RAG.

We combine:

- Instructions for the AI.
- Retrieved document content.

Example generated system prompt:

```
None
You are ScalerGPT, a helpful teaching assistant.

Answer using ONLY the context below.
If it does not contain the answer, say you don't know.

CONTEXT:

Docker is a containerization platform.

---

Docker containers package applications and dependencies.

---

Docker images are used to create containers.
```

Why use a system prompt?

It defines the behavior of the AI assistant.

For example:

```
None
Answer using ONLY the context below.
```

This instructs GPT to answer based on the retrieved content rather than relying on its general knowledge.

Important senior-level point: A system prompt is an instruction, not a security guarantee. For production applications, retrieved documents should be treated as untrusted input, and the model should be instructed not to follow instructions contained inside those documents.

14. Generate – Call GPT-4o-mini

Your code:

```
None
resp = llm.chat.completions.create(    model="gpt-4o-mini",
messages=[        {"role": "system", "content": system_prompt},
{"role": "user", "content": q.query}    ])
```

This is the Generation stage.

Let's understand the parameters.

Model

```
None
model="gpt-4o-mini"
```

Specifies the OpenAI language model.

Messages

You send two messages.

Message 1 – System

```
None
{    "role": "system",    "content": system_prompt}
```

Contains the assistant's instructions and retrieved context.

Message 2 – User

```
None
{    "role": "user",    "content": q.query}
```

Contains the original user question.

GPT receives both messages and generates an answer.

Retrieve the generated answer

```
None
resp.choices[0].message.content
```

Explanation:

- `resp`: Complete response from OpenAI.
- `choices`: List of generated completion choices.
- `[0]`: Select the first choice.
- `message`: Generated chat message.
- `content`: Actual answer text.

15. Return the final API response

```
None
return {    "question": q.query,    "answer":
resp.choices[0].message.content,    "sources_used":
len(documents[0])}
```

This returns a JSON response to the user.

Example:

```
None
{
  "question": "What is Docker?",
  "answer": "Docker is an open-source containerization platform
that packages applications and their dependencies into
containers.",
  "sources_used": 3
}
```

Here:

- `question`: Original question.
- `answer`: GPT-generated response.
- `sources_used`: Number of retrieved document chunks.

Note that `sources_used` means the number of retrieved chunks, not necessarily the number of distinct source documents.

16. Complete RAG workflow – Remember this for interviews

Three stages of RAG

1. RETRIEVE

Find relevant information

```
None
hits = collection.query(    query_texts=[q.query],    n_results=3)
```

Search ChromaDB for relevant document chunks.

2. AUGMENT

Add context to the prompt

```
None
context = "\n\n---\n\n".join(documents[0])
```

Combine retrieved information with the system prompt.

3. GENERATE

Generate the final answer

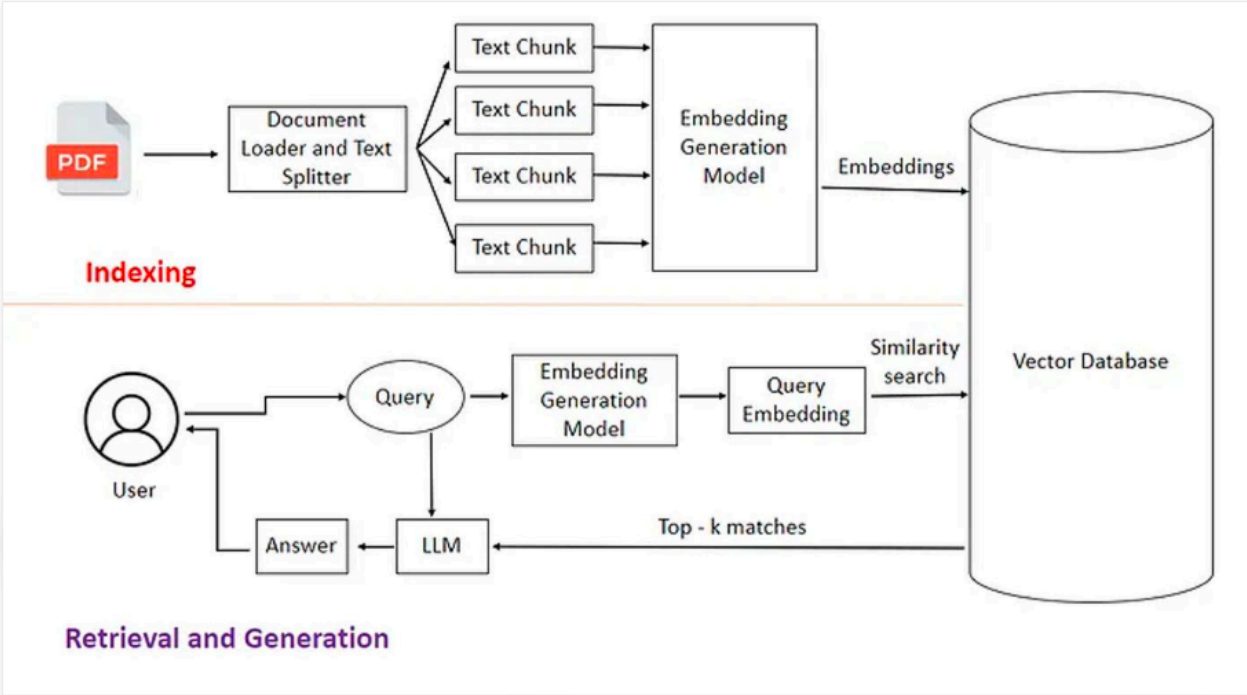
```
None
resp = llm.chat.completions.create(    model="gpt-4o-mini",
messages=[        {"role": "system", "content": system_prompt},
{"role": "user", "content": q.query}    ])
```

GPT generates an answer using the supplied context.

17. How does this connect with your previous Chat with PDF project?

There are two separate processes in a RAG application.

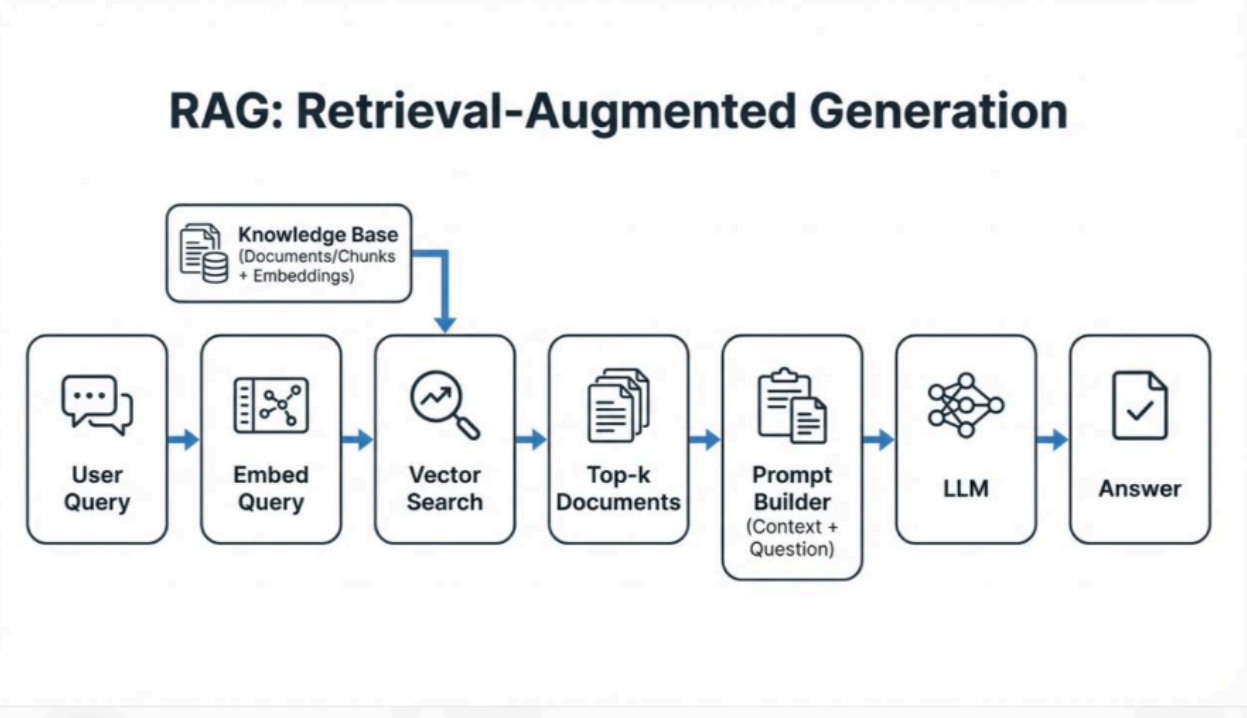
A. Document ingestion – Usually `ingest.py`



[Building a (RAG) System with Spring Boot : A Step-by-Step Guide | by Anas Elkacemi | Javarevisited | Medium](medium.com)

1. Read PDF.
2. Extract text.
3. Split text into chunks.
4. Generate embeddings.
5. Store embeddings and document chunks in ChromaDB.

B. Question answering – Your current code



[The Main Types of RAG (with graphs & illustrations)
linkedin.com]

1. Receive user question.
2. Generate question embedding.
3. Retrieve relevant chunks from ChromaDB.
4. Construct the prompt.
5. Generate an answer using GPT.
6. Return JSON response.

The important thing is that ingestion happens before question answering. Your current FastAPI code handles question answering; it does not upload or index PDFs.

18. Senior-level improvements to consider

Your implementation is a good learning project. Before using it in production, consider these improvements.

Area	Current implementation	Recommended improvement

API key	Checks placeholder	Validate configuration securely
Retry	Fixed 30 retries	Exponential backoff
Health check	Checks Chroma count	Add readiness and dependency checks
Retrieval	Top 3 chunks	Configurable top-K and relevance threshold
Prompt	Context inserted directly	Add prompt-injection defenses
Sources	Returns count only	Return source filenames and page numbers
API	Synchronous endpoints	Consider async endpoints for I/O workloads
Errors	Limited exception handling	Handle OpenAI and ChromaDB failures
Performance	Repeated collection count	Cache or optimize where appropriate
Security	No authentication	Add authentication, authorization and rate limiting

One particularly important issue: your FastAPI app uses port 8000 by default in the configuration, and ChromaDB also uses port 8000 in the default configuration. These are fine if they run in separate containers, but you must publish/map the ports carefully to avoid host-port conflicts.

19. Test your application

Once your containers are running, test these endpoints.

GET – Health check

```
None
curl http://localhost:8000/
```

POST – Ask a question

```
None
curl -X POST http://localhost:8000/ask \
  -H "Content-Type: application/json" \
  -d '{"query": "What is Docker?"}'
```

You can also open Swagger:

Open FastAPI Swagger documentation

20. Check your understanding – Interview practice

Try these five questions. Answers are below so you can revise independently.

Five interview questions

0/5 answers revealed

Q1. What is the difference between an embedding model and an LLM?

Reveal answer

Q2. Why do we use ChromaDB in this project?

Reveal answer

Q3. Why is retry logic needed when connecting to ChromaDB?

Reveal answer

Q4. What is the difference between Retrieve, Augment and Generate?

Reveal answer

Q5. What is the purpose of Pydantic BaseModel?

Reveal answer
Reset answers

My recommendation: The next important step is to understand your `ingest.py` file. Once you understand both `ingest.py` and this FastAPI code, you will be able to explain the complete RAG architecture in a senior-level Java/Python or AI Engineering interview.

I can explain your `ingest.py` line by line next, including PDF extraction, text chunking, embedding generation, and storing vectors in ChromaDB.

ingest.py

```
"""
Loads every .txt and .md file from docs/ into the Chroma vector database.

Run this AFTER the containers are up:
    docker compose exec app python ingest.py
"""
import glob
import os
import sys
import time

import chromadb
from chromadb.utils import embedding_functions

API_KEY = os.getenv("OPENAI_API_KEY", "").strip()
if not API_KEY or API_KEY.startswith("sk-paste"):
    sys.exit("[ingest] OPENAI_API_KEY is missing. Put a real key in .env and recreate.")

openai_ef = embedding_functions.OpenAIEmbeddingFunction(
    api_key=API_KEY,
    model_name="text-embedding-3-small",
)

host = os.getenv("CHROMA_HOST", "localhost")
port = int(os.getenv("CHROMA_PORT", "8000"))
```

```
chroma = None
for attempt in range(10):
    try:
        chroma = chromadb.HttpClient(host=host, port=port)
        chroma.heartbeat()
        break
    except Exception:
        print(f"[ingest] waiting for chroma ({attempt + 1}/10)...", flush=True)
        time.sleep(2)
if chroma is None:
    sys.exit(f"[ingest] Could not reach chroma at {host}:{port}")

collection = chroma.get_or_create_collection(name="notes", embedding_function=openai_ef)

files = sorted(glob.glob("docs/*.txt") + glob.glob("docs/*.md"))
if not files:
    sys.exit("No files found in docs/ — add some .txt or .md notes first.")

ids, documents, metadatas = [], [], []
for path in files:
    with open(path, "r", encoding="utf-8") as f:
        text = f.read().strip()

    # Simple chunking: split on blank lines, keep chunks of reasonable size
    chunks = [c.strip() for c in text.split("\n\n") if len(c.strip()) > 40]

    for i, chunk in enumerate(chunks):
        ids.append(f"{os.path.basename(path)}::{i}")
        documents.append(chunk)
        metadatas.append({"source": os.path.basename(path)})

if not documents:
    sys.exit("Files found but no usable chunks. Make paragraphs a bit longer.")

# upsert = add or overwrite, so re-running this script is safe
collection.upsert(ids=ids, documents=documents, metadatas=metadatas)

print(f"Ingested {len(documents)} chunks from {len(files)} file(s) ✅")
print(f"Collection now holds {collection.count()} chunks total.")
```

ScalerGPT – ingest.py Complete Explanation (Line by Line)

Your `ingest.py` file is responsible for reading documents, splitting them into smaller chunks, generating embeddings, and storing them in ChromaDB (Vector Database).

This is the first major stage of your RAG application: Document Ingestion Pipeline.

Let's understand each line with practical examples and Java comparisons.

1. Understand the complete workflow

Your project has two Python files:

File	Responsibility
`ingest.py`	Reads documents and stores their embeddings in ChromaDB
`main.py`	Accepts questions, retrieves relevant chunks and generates answers

Document ingestion architecture

docs/ folder

docker.txt, python.md, java.txt

1. Read files

glob + open()

2. Split into chunks

Split paragraphs into smaller pieces

3. Generate embeddings

OpenAI text-embedding-3-small

4. Store in ChromaDB

IDs + Documents + Metadata + Vectors

Documents ready for RAG search

2. Understand the documentation comment

```
None
"""Loads every .txt and .md file from docs/ into the Chroma vector database.Run this AFTER the containers are up:    docker compose exec app python ingest.py"""
```

This is a Python multiline string used as a module docstring.

It explains the purpose of the script.

What does this command mean?

```
None
docker compose exec app python ingest.py
```

Let's break it down.

Command	Meaning
`docker compose`	Interacts with Docker Compose

`exec`	Executes a command inside an already-running container
`app`	Name of the application service
`python`	Runs Python
`ingest.py`	Executes the ingestion script

Why execute inside the app container?

Because your app container and ChromaDB container communicate over the Docker Compose network.

This allows the ingestion script to access ChromaDB using its service name.

3. Import libraries

```
None
import globimport osimport sysimport timeimport chromadbfrom chromadb.utils import embedding_functions
```

Let's understand each library.

Library	Responsibility	Example
`glob`	Finds files matching patterns	`docs/*.txt`
`os`	Reads environment variables and file paths	`os.getenv()`

`sys`	Exits the program	`sys.exit()`
`time`	Adds delay	`time.sleep(2)`
`chromadb`	Connects to vector database	`HttpClient()`
`embedding_functions`	Generates embeddings using OpenAI	`OpenAIEmbeddingFunction()`

Java comparison

In Java, you might use:

- `Files.list()` to discover files.
- `System.getenv()` to read environment variables.
- `Thread.sleep()` to pause execution.
- A ChromaDB client library to connect to the vector database.

4. Read and validate OpenAI API key

```
None
API_KEY = os.getenv("OPENAI_API_KEY", "").strip()
```

This reads the API key from the environment.

For example, your `.env` file:

```
None
OPENAI_API_KEY=sk-proj-xxxxxxxx
CHROMA_HOST=chroma
CHROMA_PORT=8000
```

Important: `.env` is not loaded automatically by `os.getenv()`. Docker Compose must inject these variables into the container, or you must explicitly load them using `python-dotenv`.

Validate the key

```
None
if not API_KEY or API_KEY.startswith("sk-paste"):
sys.exit("[ingest] OPENAI_API_KEY is missing. Put a real key in
.env and recreate.")
```

Two validations:

1. API key must not be empty.
2. API key must not contain the placeholder prefix `sk-paste`.

If validation fails, the program terminates.

Concept: Fail Fast

We validate critical configuration before starting the actual business logic.

This prevents confusing errors later during embedding generation.

5. Initialize OpenAI embedding function

```
None
openai_ef = embedding_functions.OpenAIEmbeddingFunction(
api_key=API_KEY,    model_name="text-embedding-3-small",)
```

Important concept

This creates the embedding function.

Remember:

Embedding means converting text into numerical vectors.

Suppose your document contains:

```
None
Docker is a containerization platform.
```

The embedding model converts the text into a numerical representation.

Conceptually:

```
None
[    0.123,    -0.456,    0.789,    0.321,    ...]
```

The actual embedding from `text-embedding-3-small` has 1,536 dimensions by default.

Why do we need embeddings?

Suppose your document contains:

```
None
Docker is a containerization platform.
```

And a user asks:

```
None
Explain container technology.
```

The exact words are different, but the meanings are similar.

Vector similarity search helps ChromaDB find relevant content.

Important distinction

Component	Responsibility
Embedding model	Converts text into vectors
ChromaDB	Stores and searches vectors
GPT-4o-mini	Generates answers

One important detail: defining `openai_ef` does not immediately generate embeddings for all files. The embedding function is invoked when documents are inserted through ChromaDB.

6. Configure ChromaDB connection

```
None
host = os.getenv("CHROMA_HOST", "localhost")port =
int(os.getenv("CHROMA_PORT", "8000"))
```

These variables identify the ChromaDB server.

Why use `localhost`?

When running Python directly on your machine, ChromaDB may be accessible through localhost.

But when running inside Docker Compose:

```
None
CHROMA_HOST=chroma
CHROMA_PORT=8000
```

Here, `chroma` is the Docker Compose service name.

Important: Your `main.py` and `ingest.py` should use the same ChromaDB connection configuration.

7. Connect to ChromaDB with retry logic

```
None
chroma = None
```

Initially, no database client is available.

We initialize it with `None`.

Why?

We need to distinguish between:

- Connection established.
- Connection not established.

Next:

```
None
for attempt in range(10):
```

This loop attempts to connect a maximum of 10 times.

Create the HTTP client

```
None
chroma = chromadb.HttpClient(    host=host,    port=port)
```

This creates the ChromaDB HTTP client.

Check connection

```
None
chroma.heartbeat()
```

This sends a heartbeat request to check whether ChromaDB is responding.

If successful

```
None
break
```

The loop stops immediately.

If unsuccessful

```
None
except Exception:    print(        f"[ingest] waiting for chroma
({attempt + 1}/10)...",        flush=True    )    time.sleep(2)
```

It:

1. Catches the connection error.
2. Prints the attempt number.
3. Waits two seconds.
4. Tries again.

What happens if all attempts fail?

```
None
if chroma is None:    sys.exit(        f"[ingest] Could not reach chroma at {host}:{port}"    )
```

The application terminates.

Example retry execution

1. Attempt 1 – ChromaDB unavailable.
2. Wait 2 seconds.
3. Attempt 2 – ChromaDB unavailable.
4. Wait 2 seconds.
5. Attempt 3 – ChromaDB available.

Connection successful

Maximum configured sleep time is 20 seconds, excluding connection-attempt overhead.

8. Create or retrieve the ChromaDB collection

```
None
collection = chroma.get_or_create_collection(    name="notes",    embedding_function=openai_ef,)
```

This creates or retrieves a collection called `notes`.

Think of a collection as a table in MySQL.

MySQL	ChromaDB

Table	Collection
Row	Document record
Primary key	ID
Text column	Document
Additional columns	Metadata
Vector	Embedding

Why `get_or_create_collection()`?

If the `notes` collection exists, retrieve it.

Otherwise, create it.

Why pass `embedding_function`?

```
None
embedding_function=openai_ef
```

When we insert documents, ChromaDB uses this function to generate their embeddings automatically.

This same embedding configuration is also used in your `main.py` when searching user questions.

Very important: Both ingestion and retrieval must use compatible embedding models and dimensions. Otherwise, vector search will not work correctly.

9. Find all text and Markdown files

```
None
files = sorted(    glob.glob("docs/*.txt") +
glob.glob("docs/*.md"))
```

This is an important part of the ingestion pipeline.

What does `glob.glob()` do?

It searches for files matching a pattern.

For example:

```
None
glob.glob("docs/*.txt")
```

Finds all `.txt` files inside the `docs` folder.

Similarly:

```
None
glob.glob("docs/*.md")
```

Finds all Markdown files.

Example folder structure

```
None
ScalerGPT/
│
├── main.py
├── ingest.py
├── docker-compose.yml
│
└── docs/
    ├── docker.txt
    ├── python.md
    ├── java.txt
    └── springboot.md
```

The result might be:

```
None
files = [    "docs/docker.txt",    "docs/java.txt",
"docs/python.md",    "docs/springboot.md"]
```

Why use `sorted()`?

It sorts file paths alphabetically.

This makes the processing order predictable.

Validate whether files exist

```
None
if not files:    sys.exit(        "No files found in docs/ — add
some .txt or .md notes first."    )
```

If no matching files exist, terminate the script.

This avoids processing an empty folder.

10. Initialize three empty lists

```
None
ids, documents, metadatas = [], [], []
```

This is one of the most important lines.

We create three lists to hold the data before inserting it into ChromaDB.

Variable	Stores	Example
`ids`	Unique document chunk IDs	`docker.txt::0`

`documents`	Actual text chunks	`Docker is a containerization platform.`
`metadatas`	Additional information	`{"source": "docker.txt"}`

Let's understand why we need all three.

Imagine you have one file:

`docs/docker.txt`

Content:

```
None
Docker is a containerization platform.

Docker images are used to create containers.

Docker containers package applications and dependencies.
```

After processing, the lists might look like this:

IDs

```
None
ids = [    "docker.txt::0",    "docker.txt::1",    "docker.txt::2"]
```

Documents

```
None
documents = [    "Docker is a containerization platform.",
"Docker images are used to create containers.",    "Docker
containers package applications and dependencies."]
```

Metadata

```
None
metadatas = [    {"source": "docker.txt"},    {"source":
"docker.txt"},    {"source": "docker.txt"}]
```

All three lists correspond by index.

For example:

```
None
ids[0]documents[0]metadatas[0]
```

Represent the same document chunk.

11. Read files one by one

```
None
for path in files:
```

This loops through every discovered file.

For example:

```
None
Iteration 1: docs/docker.txt
Iteration 2: docs/java.txt
Iteration 3: docs/python.md
Iteration 4: docs/springboot.md
```

Open the file

```
None
with open(path, "r", encoding="utf-8") as f:    text =
f.read().strip()
```

Let's break this down.

```
open(path, "r")
```

Opens the file in read mode.

`encoding="utf-8"`

Ensures text is decoded using UTF-8 encoding.

`with`

Automatically closes the file after processing, even if an exception occurs.

`f.read()`

Reads the complete file content into memory as a string.

`.strip()`

Removes whitespace from the beginning and end.

Example:

Before:

```
None

    Docker is a containerization platform.
```

After:

```
None
Docker is a containerization platform.
```

12. Split the document into chunks

Very important RAG concept

Your code:

```
None
chunks = [    c.strip()    for c in text.split("\n\n")    if
len(c.strip()) > 40]
```

This is called Text Chunking.

Why do we need chunking?

Imagine a document contains 100 pages.

Sending the entire document every time someone asks a question is inefficient.

Instead, divide the document into smaller pieces.

Then retrieve only the relevant pieces.

Understand `text.split("\n\n")`

It splits the text wherever there are two consecutive newline characters.

Example:

Original document:

```
None
Docker is a containerization platform.

Docker images are used to create containers.

Docker containers package applications and dependencies.
```

After splitting:

```
None
[    "Docker is a containerization platform.",    "Docker images
are used to create containers.",    "Docker containers package
applications and dependencies."]
```

Understand list comprehension

The expression:

```
None
[    c.strip()    for c in text.split("\n\n")    if len(c.strip())
> 40]
```

Can also be written as:

```
None
chunks = []for c in text.split("\n\n"):    cleaned_chunk =
c.strip()    if len(cleaned_chunk) > 40:
chunks.append(cleaned_chunk)
```

Both approaches perform the same operation.

Why `len(c.strip()) > 40`?

It filters out paragraphs that contain 40 or fewer characters.

For example:

```
None
Hi
```

This paragraph is ignored.

A longer paragraph is retained.

Important limitation: This is simple paragraph-based chunking, not advanced chunking. If a document has a very long paragraph, it will remain one very large chunk.

For production, consider:

- Recursive character text splitting.
- Token-based chunking.
- Chunk overlap.
- Maximum chunk size.

For example, you might use 500 tokens per chunk with 50 tokens of overlap, depending on your document type.

13. Generate unique IDs for each chunk

```
None
for i, chunk in enumerate(chunks):
```

Here, `enumerate()` gives you two values:

- `i`: Index of the chunk.
- `chunk`: Actual text.

Example:

```
None
chunks = [    "Docker is a containerization platform.",    "Docker images create containers.",    "Docker containers package applications."]
```

Execution:

i	chunk
0	Docker is a containerization platform.
1	Docker images create containers.
2	Docker containers package applications.

Create unique ID

```
None
ids.append(f"{os.path.basename(path)}::{i}")
```

Let's understand both methods.

`os.path.basename(path)`

Extracts the filename from the full path.

Example:

```
None
path = "docs/docker.txt"os.path.basename(path)
```

Output:

```
None
docker.txt
```

`f"{...}"`

This is Python string interpolation, similar to Java's formatted strings.

Final ID:

```
None
docker.txt::0
docker.txt::1
docker.txt::2
```

Why is this useful?

Every chunk needs a unique identifier in ChromaDB.

Add actual text

```
None
documents.append(chunk)
```

Stores the actual paragraph in the documents list.

Add metadata

```
None
metadatas.append({    "source": os.path.basename(path)})
```

Stores the original filename associated with the chunk.

For example:

```
None
{    "source": "docker.txt"}
```

Why is metadata useful?

Suppose a user asks:

```
None
What is Docker?
```

You retrieve a relevant chunk.

You can also identify the original source file from its metadata.

This is useful for displaying references and citations.

14. Validate whether usable chunks exist

```
None
if not documents:    sys.exit(        "Files found but no usable
chunks. Make paragraphs a bit longer."    )
```

You might have files inside `docs/`, but all paragraphs could be too short.

For example:

```
None
Hi.

Hello.
```

```
Test.
```

All paragraphs fail:

```
None
len(c.strip()) > 40
```

Therefore, `documents` remains empty.

The script terminates with a meaningful message.

15. Store documents in ChromaDB using upsert

Most important database operation

```
None
collection.upsert(    ids=ids,    documents=documents,
metadatas=metadatas)
```

This is where your document ingestion actually happens.

Let's understand the three arguments.

Argument	Value
`ids`	Unique chunk IDs
`documents`	Text content

`metadatas`	Source information

ChromaDB stores these records and generates embeddings using the configured `openai_ef`.

Conceptually, one record looks like this:

ChromaDB record

ID: docker.txt::0

Document

Docker is a containerization platform.

Metadata

```
None
{
  "source": "docker.txt"
}
```

Embedding

A 1,536-dimensional numerical vector generated by OpenAI.

What is upsert?

`upsert` means:

- If the ID does not exist, insert a new record.
- If the ID already exists, update/overwrite that record.

Example:

First execution:

```
None
docker.txt::0 → Insert
docker.txt::1 → Insert
```

```
docker.txt::2 → Insert
```

Second execution with the same IDs:

```
None
docker.txt::0 → Update
docker.txt::1 → Update
docker.txt::2 → Update
```

This makes rerunning the script safe from duplicate IDs.

Important senior-level observation: `upsert` does not delete old records whose IDs are no longer generated. For example, if a file previously had 10 chunks and now has 6, the previous chunks numbered 6–9 can remain in ChromaDB. A production ingestion pipeline should reconcile deleted or obsolete IDs.

16. Print ingestion summary

```
None
print(    f"Ingested {len(documents)} chunks from {len(files)}
file(s) ✅")
```

This prints how many chunks were processed and how many files were read.

Example:

```
None
Ingested 25 chunks from 4 file(s) ✅
```

Print total ChromaDB records

```
None
print(    f"Collection now holds {collection.count()} chunks
total.")
```

This returns the total number of records in the collection, including any records already stored before the current run.

Example:

```
None
Collection now holds 25 chunks total.
```

If there were previously 10 records from another ingestion run, the final count could be higher.

17. Complete execution example

Let's take a practical example.

Your folder:

```
None
docs/
├── docker.txt
└── python.md
```

`docker.txt`:

```
None
Docker is a containerization platform.

Docker images are used to create containers.
```

`python.md`:

```
None
Python is a high-level programming language.

Python supports object-oriented programming.
```

Assuming each paragraph contains more than 40 characters, your ingestion script creates four chunks.

Expected ingestion output

Successful ingestion

```
None
Ingested 4 chunks from 2 file(s) ✅

Collection now holds 4 chunks total.
```

Stored records

ID	Source
docker.txt::0	docker.txt
docker.txt::1	docker.txt
python.md::0	python.md
python.md::1	python.md

Now the user can ask:

```
None
{
  "query": "What is a Docker image?"
}
```

Your `main.py` searches the `notes` collection and retrieves relevant chunks.

GPT then generates the answer using those chunks.

18. Important senior-level improvements

Your current code is good for learning RAG. Here are some improvements worth understanding.

Area	Current implementation	Improvement
File support	`.txt`, `.md`	Add PDF/DOCX loaders if required
Chunking	Split by blank lines	Recursive/token-based chunking
Chunk size	Only minimum 40 characters	Configure maximum token size
Overlap	No overlap	Add overlapping chunks
IDs	Filename + index	Consider content hashes or stable IDs
Metadata	Source filename	Add page number, title, timestamps
Updates	Upsert only	Remove obsolete chunks
Errors	Basic connection retry	Add structured exception handling
Large files	Reads everything into memory	Stream or process files incrementally
Embeddings	One embedding configuration	Add cost and rate-limit management

One more important point: `collection.upsert()` sends the records for insertion. ChromaDB calls the embedding function to create vectors for the documents that need to be inserted or updated. You do not manually create vectors in this script.

19. Interview questions – Test your understanding

RAG interview practice

0/5 revealed

1. What is document ingestion in RAG?

Reveal answer

2. Why do we split documents into chunks?

Reveal answer

3. What is the difference between add() and upsert() in ChromaDB?

Reveal answer

4. Why do we store metadata along with document chunks?

Reveal answer

5. What happens if the embedding model used during ingestion differs from the model used during retrieval?

Reveal answer
Reset answers

20. Final revision – Understand both files together

This is the most important distinction to remember for your AI Engineering interviews.

Concept	`ingest.py`	`main.py`
Purpose	Prepare documents	Answer questions
Input	`.txt` and `.md` files	User question
Chunking	Yes	No
Embedding	Document embeddings	Query embedding
ChromaDB	Stores documents	Searches documents
GPT	Not required	Generates answer
Output	Stored vector records	JSON answer

Remember this simple explanation:

- `ingest.py` teaches the vector database what your documents contain.
- `main.py` searches that knowledge when a user asks a question.
- GPT uses the retrieved information to generate the final response.

This is the complete foundation of your ScalerGPT RAG application.